

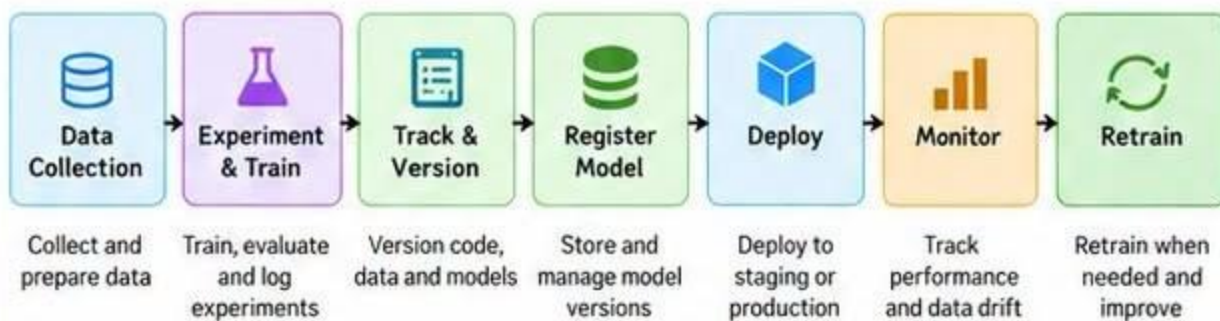
MLOps

Complete Cheat Sheet

Build • Deploy • Monitor • Scale • Repeat

improve

1. MLOps Workflow (Big Picture)



2. MLOps Tools Ecosystem



3. Experiment Tracking

Log parameters, metrics, and artifacts to compare experiments.

mlflow

```
import mlflow
mlflow.set_experiment("iris")

with mlflow.start_run():
    mlflow.log_param("lr", 0.01)
    mlflow.log_param("epochs", 10)
    mlflow.log_metric("accuracy", 0.95)
    mlflow.log_artifact("model.pkl")
```

- ✓ Compare multiple runs
- ✓ Visualize metrics and artifacts
- ✓ Reproduce results

4. Versioning (Code, Data, Models)

Version everything for reproducibility.

git

```
# Git for code versioning
git init
git add .
git commit -m "Initial commit"
git push origin main
```

DVC

```
# DVC for data and model versioning
dvc add data/
dvc add model.pkl
git add data.dvc model.pkl.dvc
git commit -m "Add data and model"
dvc push
```

- ✓ Track changes
- ✓ Collaborate easily
- ✓ Reproduce experiments

5. Model Registry

Store, manage and version trained models.

mlflow

```
# After logging the model
model_uri = "runs:/<run_id>/model"

mlflow.register_model(
    model_uri,
    "iris-classifier"
)
```

- ✓ Manage model versions
- ✓ Track model lineage
- ✓ Use aliases & tags (e.g. champion)
- ✓ Rollback if needed

6. CI Pipeline for ML

Automate testing and build checks.

GitHub Actions

```
# .github/workflows/ci.yml
name: CI
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v5
      - with:
        python-version: '3.10'
      - run: pip install -r requirements.txt
      - run: pip install build
      - run: python -m build
      - run: pytest
```

- ✓ Automate testing
- ✓ Run tests on code push
- ✓ Verify build succeeds

7. Deployment

Deploy models as APIs or scalable services.

FastAPI **Docker**

```
# Simple FastAPI deployment
from fastapi import FastAPI
import joblib

app = FastAPI()
model = joblib.load("model.pkl")

@app.post("/predict")
def predict(data: dict):
    pred = model.predict(data["features"])
    return {"prediction": int(pred[0])}
```

- ✓ Serve models as REST APIs
- ✓ Use Docker for containerization
- ✓ Deploy on cloud (AWS, GCP, Azure)

8. Monitoring

Track model performance and data quality in production.

Evidently AI **Grafana**

```
# Detect data drift with Evidently
from evidently import Dataset, DataDefinition, Report
from evidently.presets import DataDriftPreset

definition = DataDefinition()
reference = Dataset.from_pandas(
    reference_df,
    data_definition=definition
)
current = Dataset.from_pandas(
    current_production_df,
    data_definition=definition
)
report = Report([DataDriftPreset()])
result = report.run(current, reference)

# Monitor predictions and model quality
# Set up alerts for drift
```

9. Retraining

Trigger retraining when performance degrades or significant data/concept drift is detected.

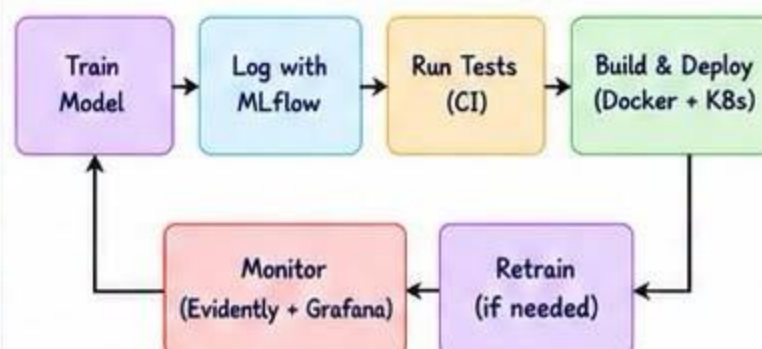
Retrain

```
# Simplified retraining workflow
if performance < threshold:
    print("Retraining model...")
    new_data = load_new_data()
    model = train_model(new_data) # your
    # training function
    acc = evaluate_model(model)
    with mlflow.start_run():
        mlflow.log_metric("new_accuracy", acc)
        mlflow.sklearn.log_model(model, "model")
        deploy_model # your deployment
```

- ✓ Set retraining triggers
- ✓ Use latest data
- ✓ Automate the pipeline
- ✓ Deploy new version seamlessly

10. MLOps Pipeline Example

End-to-end pipeline from training to monitoring.



11. Useful Commands

Build Docker Image

```
docker build -t my-ml-app .
```

Run Docker Container

```
docker run -p 8000:8000 my-ml-app
```

Deploy to Kubernetes

```
kubectl apply -f deployment.yaml
```

Check Model in MLflow

```
mlflow models serve --runs:/<run_id>/model \
--port 5000
```

View Monitoring Dashboard

```
grafana-server
```



MLOps = Reliable AI in the Real World

Track • Automate • Monitor • Improve

Faster Experiments

Reliable Deployment

Better Decisions

Real Impact